



CL1002-Programming Fundamentals— FALL 2023

LAB 13

Multi-Dimensional Arrays



Learning Outcomes

In this lab you are expected to learn the following:

- 2D Arrays
- Pass by Value and Pass by Reference

Multi-Dimensional Array:

A multi-dimensional array is an array of arrays.

Declaring a Multi-Dimensional Array:

To declare a multi-dimensional array, define the variable type, specify the name of the array followed by square brackets which specify how many elements the main array has, followed by another set of square brackets which indicates how many elements the sub-arrays have.

For example:

- `int array [3][4];`

	0	1	2	3
0	[0][0]	[0][1]	[0][2]	[0][3]
1	[1][0]	[1][1]	[1][2]	[1][3]
2	[2][0]	[2][1]	[2][2]	[2][3]

- `char Names [2][3]={`
 `{ 'A', 'L', 'I' },`
 `{ 'Z', 'I', 'A' }`
 `};`

Access the Elements of a Multi-Dimensional Array:

To access an element of a multi-dimensional array, specify an index number in each of the array's dimensions. This statement accesses the value of the element in the first row (0) and third column (2) of the letters array.

For example:

- `string letters = {`
 `{ "A", "B", "C", "D" },`
 `{ "E", "F", "G", "H" }`
 `};`

 `cout << letters[0][2];    // Outputs "C"`

Change Elements in a Multi-Dimensional Array:

To change the value of an element, refer to the index number of the element in each of the dimensions.

For example:

```
• string letters[2][4] = {  
    { "A", "B", "C", "D" },  
    { "E", "F", "G", "H" }  
};  
letters[0][0] = "Z";  
cout << letters[0][0]; // Now outputs "Z" instead of "A"
```

Loop Through a Multi-Dimensional Array:

To loop through a multi-dimensional array, you need one loop for each of the array's dimensions. The following example outputs all elements in the letters array.

For example:

```
• string letters[2][4] = {  
    { "A", "B", "C", "D" },  
    { "E", "F", "G", "H" }  
};  
  
for (int i = 0; i < 2; i++) {  
    for (int j = 0; j < 4; j++) {  
        cout << letters[i][j] << "\n";  
    }  
}
```

Example 01: Using Two-Dimensional Array

```
1  #include <iostream>  
2  using namespace std;  
3  int main() {  
4      int test[3][2] = {{2, -5}, {4, 0}, {9, 1}};  
5      // use of nested for loop  
6      // access rows of the array  
7      for (int i = 0; i < 3; ++i) {  
8          // access columns of the array  
9          for (int j = 0; j < 2; ++j) {  
10             cout << "test[" << i << "][" << j << "] = " << test[i][j] << endl; }  
11     }
```



```
12 return 0;
13 }
```

Output:

```
test[0][0] = 2
test[0][1] = -5
test[1][0] = 4
test[1][1] = 0
test[2][0] = 9
test[2][1] = 1
```

Example 02: Passing array as an argument to a function

```
1 #include <iostream>
2 using namespace std;
3 void displayArray(int array[], int size){
4     for(int i=0; i<size; i++){
5         cout<<array[i]<<" ";
6     }
7     cout<<endl;
8 }
9 int main()
10 {
11     int theArray[]={2,4,6,7,8,9};
12     int theSize = sizeof(theArray)/sizeof(theArray[0]);
13     displayArray(theArray, theSize);
14 }
```

Output:

```
2 4 6 7 8 9
```

Difference in Passing by Value and Passing by Reference:

Passing by Value:

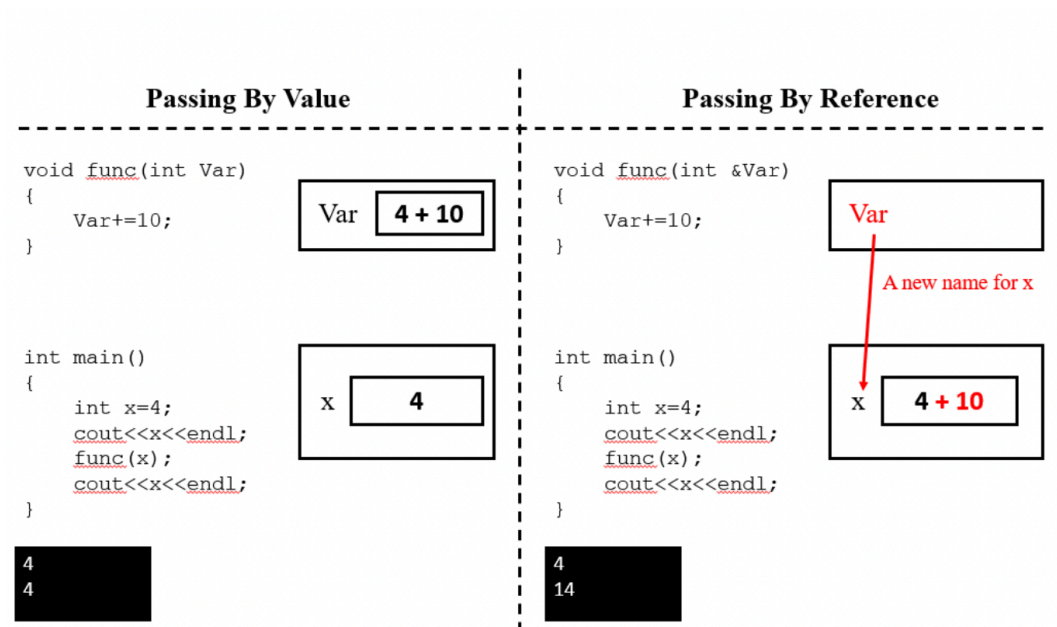
A variable (suppose its value is 4) is passed to a function,

- a usual parameter of the function catches the value
- the function adds 10 to its parameter (parameter's value)
- it won't affect the value of the original variable passed from the main function
- a separate memory location has already allocated to the original variable in main memory
- a separate memory location will be occupied by the parameter in the function

Passing by Reference:

A variable (suppose its value is 4) is passed to a function,

- a different parameter is used here this time
- address/reference operator followed by parameter name(¶m)
- the function also adds 10 to its parameter
- it shall change the value of the original variable passed from the main function without using the return keyword
- the parameter in this particular case will act as another, second label (variable) for the same memory location of the variable passed from the main function



Submission Instructions:

1. Save the **cpp** file with the roll no and task number
e.g. i230001_Q1.cpp
2. Now create a new folder with **ROLLNO_LAB11_SEC** e.g. i23XXXX_LAB13_A
3. You need to display your roll no and name before the output of each question.
4. Now you have to submit this zipped file on Google Classroom.
5. If you don't follow the above-mentioned submission instructions, you will be marked **zero**.
6. Plagiarism in the Lab Task will result in **zero** marks in the whole category.



Lab Tasks

Task 01

Write a function named **decreaseSort()** which takes an array and its size as arguments and sorts an array of integers in decreasing order. The **main** function should use the functions to sort it in decreasing order and display the sorted array. Declare an array of a fixed size 6 and take input from the user.

Original Array: 9 4 7 2 5 1

Sorted Array (Decreasing Order): 9 7 5 4 2 1

Task 02

Write a function named **removeDuplicates()** which takes a sorted array and its size as arguments and deletes any duplicate number from that array. The array would be passed from the main function with values from the user and the constant size of 8 elements. The final output should be displayed in the main function.

Sample Output:

Original Array

2	4	4	5	6	7	7	7
---	---	---	---	---	---	---	---

Updated Array

2	4	5	6	7
---	---	---	---	---

Task 03

Write a C++ program that takes a matrix as input, and finds its transpose using a function named **transpose()**. The function would take the array and its size as arguments. Declare an array of constant rows 2 and cols 3 and pass it to the function that would calculate the transpose of an array, store it in an array, and display it inside the function. The return type of function would be void.

Sample Output:



Original Matrix:

1 2

3 4

5 6

Transpose:

1 3 5

2 4 6

Task 04

Write a C++ function named **Spiral()** that takes an array and its rows and cols as arguments and prints the array in spiral form. Declare an array of constant rows = 4 and cols = 4 in the main and take values as input from the user.

Matrix:

1	→	2	→	3	→	4	↓
5	→	6	→	7	↓	8	↓
↑	9	10	←	11	↓	12	↓
↑	13	←	14	←	15	←	16

Output:

1, 2, 3, 4, 8, 12, 16, 15, 14, 13, 9, 5, 6, 7, 11, 10